

# Tutorial: Agile Hardware Design with Chisel

Martin Schoeberl

Technical University of Denmark  
Embedded Systems Engineering

September 10, 2026

# Motivating Example:

## Lipsi: Probably the Smallest Processor in the World

- ▶ Tiny processor
- ▶ Simple instruction set
- ▶ Shall be small
  - ▶ Around 200 logic cells, one FPGA memory block
- ▶ Hardware described in Chisel
- ▶ Available at <https://github.com/schoeberl/lipsi>
- ▶ Usage
  - ▶ Utility processor for small stuff
  - ▶ In teaching introduction to computer architecture
- ▶ The design took place on the island of Lipsi

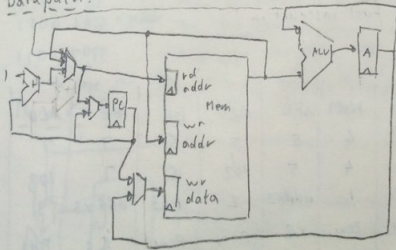
# The Design of Lipsi on Lipsi

## Lipsi: a Minimalistic Microcontroller

Levos, Lipsi  
16.7.2010 81

- Single on-chip memory  $\Rightarrow$  2 cycles/instruction
- $\approx 200$  Ld! (ld reg indirect is 3 cycles)
- 8 bit datapath, 8 bit variable length instructions
- Accu + 8 (16) register in memory
- 256 byte instructions, 256 byte data

Datapath:



# Lipsi Implementation

- ▶ Hardware described in Chisel
- ▶ Tester in Chisel/Scala
- ▶ Assembler in Scala
  - ▶ Core case statement about 20 lines
- ▶ Reference design of Lipsi as a software simulator in Scala
- ▶ Testing:
  - ▶ Self-testing assembler programs
  - ▶ Comparing hardware with a software simulator
- ▶ All in a single programming language!
- ▶ All in a single program
- ▶ How much work is this?

# Chisel is Productive

- ▶ All coded and tested in less than 14 hours!
- ▶ The hardware in Chisel
- ▶ Assembler in Scala
- ▶ Some assembler programs (blinking LED)
- ▶ Simulation in Scala
- ▶ Two testers
- ▶ BUT, this does not include the design (done on paper)

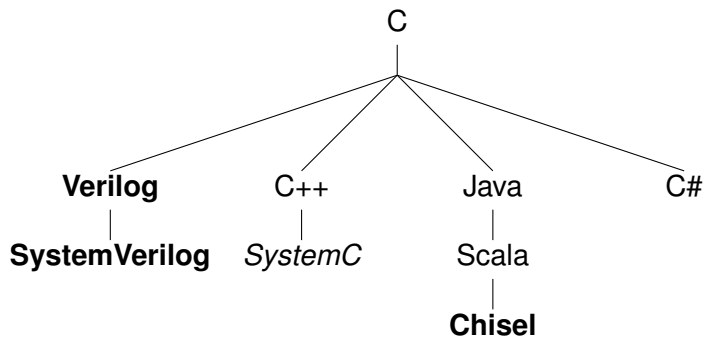
# Motivating Example: Lipsi, a Tiny Processor

- ▶ Show in IntelliJ

# Chisel

- ▶ A hardware *construction* language
  - ▶ Constructing Hardware in a Scala Embedded Language
  - ▶ If it compiles, it is synthesizable hardware
  - ▶ Say goodbye to your unintended latches
- ▶ Chisel is not a high-level synthesis language
- ▶ Single source for two targets
  - ▶ Cycle-accurate simulation (testing)
  - ▶ Verilog for synthesis
- ▶ Embedded in Scala
  - ▶ Full power of Scala available
  - ▶ We use Scala to write the generators
- ▶ Developed at UC Berkeley

# The C Language Family



# Other Language Families

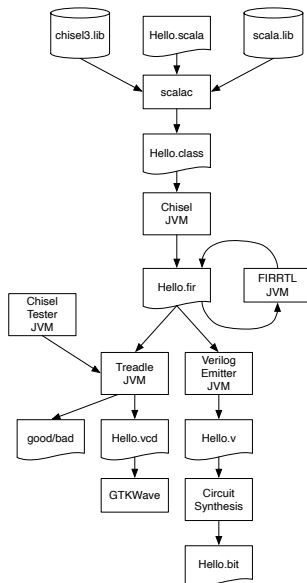
Algol  
|  
Ada  
|  
**VHDL**

Python  
|  
**MyHDL**

# A Small Language

- ▶ Chisel is a *small* language
- ▶ On purpose
- ▶ Not many constructs to remember
- ▶ The [Chisel Cheatsheet](#) fits on two pages
- ▶ The power comes with Scala for circuit generators
- ▶ With Scala, Chisel can grow with you

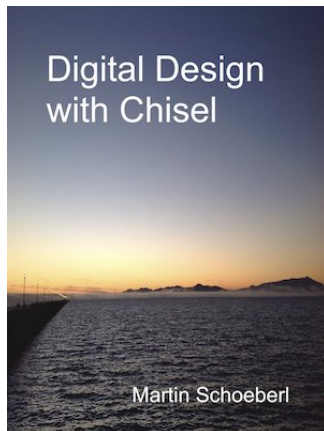
# Tool Flow for Chisel Defined Hardware



# Chisel is a Hardware Construction Language

- ▶ The code looks much like Java code
- ▶ But it is *not* a program in the usual sense
- ▶ It represents a circuit
- ▶ The “program” constructs the circuit
- ▶ All statements are “executed” in parallel
- ▶ Statement order has *mostly* no meaning

# A Chisel Book



- ▶ Available in open access (as PDF)
  - ▶ Optimized for reading on a tablet (size, hyperlinks)
- ▶ Amazon can do the printout

# Functions

- ▶ Circuits can be encapsulated in functions
- ▶ Each *function call* generates hardware
- ▶ Simple functions can be a single line
- ▶ Functions can include state (registers)

```
def adder(v1: UInt, v2: UInt) = v1 + v2
```

```
val add1 = adder(a, b)
```

```
val add2 = adder(c, d)
```

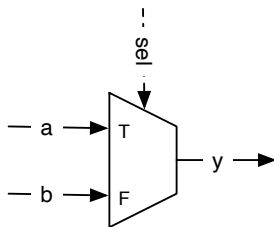
# Conditional Circuit Generation

```
class Base extends Module { val io = new Bundle() }  
class VariantA extends Base { }  
class VariantB extends Base { }
```

```
val m = if (useA) Module(new VariantA())  
        else Module(new VariantB())
```

- ▶ if and else is Scala
- ▶ if is an expression that returns a value
  - ▶ Like “cond ? a : b;” in C and Java
- ▶ This is not a hardware multiplexer
- ▶ Decides which module to generate
- ▶ Could even read an XML file for the configuration

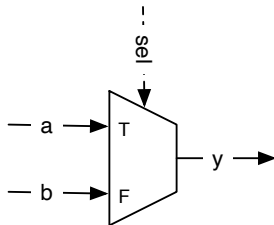
# Chisel has a Multiplexer



```
val result = Mux(sel, a, b)
```

- ▶ So what?
- ▶ Wait... What type is a and b?
  - ▶ Can be any Chisel type!

# Chisel has a Generic Multiplexer



```
val result = Mux(sel, a, b)
```

- ▶ SW people may not be impressed
- ▶ They have generics since Java 1.5 in 2004
  - ▶ `List<Flowers> != List<Cars>`

# Generics in Hardware Construction

- ▶ Chisel supports generic classes with type parameters
- ▶ Write hardware generators independent of concrete type
- ▶ This is a multiplexer *generator*

```
def myMux[T <: Data](sel: Bool, tPath: T, fPath:
    T): T = {

    val ret = WireDefault(fPath)
    when (sel) {
        ret := tPath
    }
    ret
}
```

# Put Generics Into Use

- ▶ Let us implement a generic FIFO
- ▶ Use the generic ready/valid interface from Chisel

```
class DecoupledIO[T <: Data](gen: T) extends
  Bundle {
    val ready = Input(Bool())
    val valid = Output(Bool())
    val bits  = Output(gen)
  }
```

# Define the FIFO Interface

```
class FifoIO[T <: Data](private val gen: T)
  extends Bundle {
    val enq = Flipped(new DecoupledIO(gen))
    val deq = new DecoupledIO(gen)
  }
```

- ▶ We need enqueueing and dequeueing ports
- ▶ Note the Flipped
  - ▶ It switches the direction of ports
  - ▶ No more double definitions of an interface

## But What FIFO Implementation?

- ▶ Bubble FIFO (good for low data rate)
- ▶ Double buffer FIFO (fast restart)
- ▶ FIFO with memory and pointers (for larger buffers)
  - ▶ Using flip-flops
  - ▶ Using on-chip memory
- ▶ And some more...
- ▶ This calls for object-oriented programming *hardware construction*

# Abstract Base Class and Concrete Extension

```
abstract class Fifo[T <: Data](gen: T, val depth:
  Int) extends Module {
  val io = IO(new FifoIO(gen))

  require(depth > 0, "Number of buffer elements
    needs to be larger than 0")
}
```

- ▶ May contain common code
- ▶ Extend by concrete classes

```
class BubbleFifo[T <: Data](gen: T, depth: Int)
  extends Fifo(gen: T, depth: Int) {
```

# Select a Concrete FIFO Implementation

- ▶ Decide at hardware generation
- ▶ Can use all Scala/Java power for the decision
  - ▶ Connect to a web service, get Google Alphabet stock price, and decide on which to use ;-)
  - ▶ For sure a silly idea, but you see what is possible...
  - ▶ Developers may find clever use of the Scala/Java power
  - ▶ We could present a GUI to the user to select from
- ▶ We use XML files parsed at hardware generation time
- ▶ End of TCL, Python,... generated hardware

# Binary to BCD Conversion for VHDL

```

public class GenBcdConv {

    static final int ADDRESS = 6;
    static final int DATASIZE = 8;
    static final int ROM_SIZE = 1 << ADDRESS;

    String bin(int val, int bits) {

        String s = "";
        for (int i = 0; i < bits; ++i) {
            s = (val & (1 << (bits - i - 1))) > 0 ? "1" : "0";
        }
        return s;
    }

    String getHeader() {

        String line = "";
        line += "\n(bcdtab.vhd)";
        line += "\n";
        line += "Generated VHDL table for BCD conversion";
        line += "\n";
        line += "DO NOT edit this file!";
        line += "\n(Generated by " + this.getClass().getName() + ")";
        line += "\n";
        line += "Library ieee;";
        line += "use ieee.std_logic_1164.all;";
        line += "\n";
        line += "entity bcdtab is";
        // line +=
        // generic (width : integer; addr_width : integer) (-- for compatibility);
        line += "port (";
        line += "    address : in std_logic_vector" + " (ADDRESS - 1)";
        line += "    q : out std_logic_vector" + " (DATASIZE - 1)";
        line += "    q1 : out std_logic_vector" + " (DATASIZE - 1)";
        line += ");";
        line += "end bcdtab;";
        line += "\n";
        line += "architecture rtl of bcdtab is";
        line += "\n";
        line += "begin";
        line += "\n";
        line += "p1<=address; begin1";
        line += "\n";
        line += "case address is";
        return line;
    }

    String getHeaderFoot() {

        String line = "\n";
        line += "    when others => q <= \"\" + bin(0, DATASIZE) + "\"";
        line += "end case";
        line += "end process";
        line += "\n";
        line += "end rtl;";
        return line;
    }

    public void dump() {

        try {
            FileWriter fw = new FileWriter("bcdtab.vhd");
            fw.write(getHeader());

            for (int i = 0; i < Math.pow(2, ADDRESS); ++i) {
                int val = ((i/8)>0) < (i/8) < 1;
                fw.write("    when \"" + bin(i, ADDRESS) + "\" => q <= \""
                    + bin(val, DATASIZE) + "\";");
                fw.write("\n");
            }

            fw.write(getHeaderFoot());
            fw.close();
        } catch (IOException e) {
            System.out.println(e.getMessage());
            System.exit(-1);
        }
    }

    // =
    // =
    // =
    public static void main(String[] args) throws Exception {
        GenBcdConv la = new GenBcdConv();
        la.dump();
    }
}

```

# Java Program

- ▶ Generates a VHDL table
- ▶ The core code is:

```
for (int i = 0; i < Math.pow(2, ADDRBITS); ++i) {  
    int val = ((i/10)<<4) + i%10;  
    // write out VHDL code for each line
```

- ▶ With all boilerplate 118 LoC

# Chisel Version of Binary to BCD Conversion

```
val table = Wire(Vec(100, UInt(8.W)))
for (i <- 0 until 100) {
  table(i) := (((i/10)<<4) + i%10).U
}
val bcd = table(bin)
```

- ▶ Directly generates the hardware table as a Vec
- ▶ At hardware construction time
- ▶ In the same language

# Use Functional Programming for Generators

```
def add(a: UInt, b: UInt) = a + b
```

```
val sum = vec.reduce(add)
```

```
val sum = vec.reduce(_ + _)
```

```
val sum = vec.reduceTree(_ + _)
```

- ▶ This is a simple example
- ▶ What about an arbitration tree with fair arbitration?

# Functional Generation

- ▶ Anonymous functions, called *function literal*

```
(param) => function body
```

- ▶ A function for a minimum search

```
val min = vec.reduceTree((x, y) => Mux(x <  
    y, x, y))
```

## Minimal Function with Index

- Was the example for Tjark's heap sort

```
class Two extends Bundle {  
  val v = UInt(w.W)  
  val idx = UInt(8.W)  
}  
  
val vecTwo = Wire(Vec(n, new Two()))  
for (i <- 0 until n) {  
  vecTwo(i).v := vec(i)  
  vecTwo(i).idx := i.U  
}  
  
val res = vecTwo.reduceTree((x, y) =>  
  Mux(x.v < y.v, x, y))
```

- We need an extra bundle to hold both values
- A for loop is not so functional

## We Can Use Tuples and zipWithIndex

- ▶ `zipWithIndex` transforms the original sequence to a sequence of tuples with second element is the index
- ▶ Use `map` to translate from Scala `Int` to Chisel `UInt`
- ▶ `reduce` does the minimum function, actually called 2 times (we could optimize this)

```
val resFun = vec.zipWithIndex
  .map((x) => (x._1, x._2.U))
  .reduce((x, y) => (Mux(x._1 < y._1, x._1,
    y._1), Mux(x._1 < y._1, x._2, y._2)))
```

- ▶ The result is a Scala `Vector` and not a Chisel `Vec`

## Solution with a Vec

- ▶ This results in a Chisel Vec

```
val scalaVector = vec.zipWithIndex
    .map((x) => MixedVecInit(x._1,
        x._2.U(8.W)))
val resFun2 = VecInit(scalaVector)
    .reduceTree((x, y) => Mux(x(0) < y(0), x,
        y))

val minVal = resFun2(0)
val minIdx = resFun2(1)
```

- ▶
- ▶ We should add a reduceTree to the Scala sequence version (TraversableOnce)

# Combinational (Truth) Table Generation

```
val arr = new Array[Bits](length)
for (i <- 0 until length) {
  arr(i) = ...
}
val rom = Vec[Bits](arr)
```

- ▶ Generate a table in a Scala array
- ▶ Use that array as input for a Chisel Vec
- ▶ Generate a logic table at hardware construction time

# Ideas for Runtime Table Generation

- ▶ Assembler in Scala/Java generates the boot ROM
- ▶ Binary to BCD conversion
- ▶ Schedule table for a TDM-based network-on-chip
- ▶ Read an ELF file with a Java library
- ▶ More ideas?

# Slides and Further Material

- ▶ Slides and a full course, including labs
- ▶ Material for the following tutorial
- ▶ The slide sources are part of the Chisel book source
- ▶ <https://github.com/schoeberl/agile-hw>
- ▶ Or Google: schoeberl github agile



# Part of ODE4EC-PIV

- ▶ Topic is part of the Chips JU project ODE4EC-PIV
- ▶ <https://ode4ec.eu/>
- ▶ We have a PhD position on agile HW at DTU
- ▶ [https://efzu.fa.em2.oraclecloud.com/hcmUI/CandidateExperience/da/sites/CX\\_2001/job/7982](https://efzu.fa.em2.oraclecloud.com/hcmUI/CandidateExperience/da/sites/CX_2001/job/7982)
- ▶ Deadline 18 September

# Summary

- ▶ Write reusable generators, not RTL
- ▶ Chisel + Scala + functional programming = HW generators
- ▶ We continue in room 1 with the tutorial
- ▶ This includes hands-on writing Scala/Chisel HW generators

# Lab 0: Hello World in Chisel

- ▶ Install all tools, see [Setup.md](#)
- ▶ You need Java 17+ and sbt
- ▶ Clone or download the repository from:
  - ▶ <https://github.com/schoeberl/agile-hw>
- ▶ Start IntelliJ or VSC and follow the instructions from the [lab0](#) page
- ▶ `sbt test`
- ▶ Explore the code; maybe change the example

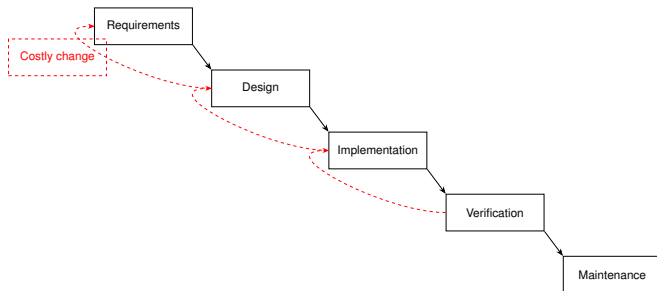
## Session 2

- ▶ Waterfall and Agile design style
- ▶ Quick overview of Chisel and Scala
- ▶ Functional programming in Scala
- ▶ Links to further reading and material

# The Classic Waterfall Model

- ▶ Traditional project management approach for software and hardware
- ▶ Development phases are sequential:
  1. Requirements
  2. Design
  3. Implementation
  4. Verification
  5. Maintenance
- ▶ Each phase must be completed before moving to the next
- ▶ Changes are costly once early phases are complete
- ▶ I worked (long time ago) in such a team: was very boring

# Waterfall Model Diagram



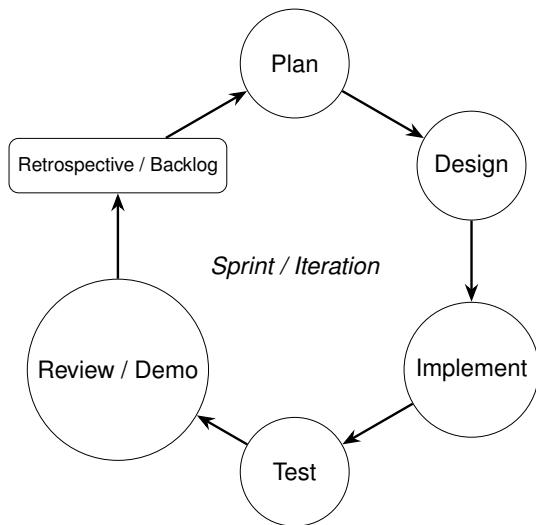
# Limitations of the Waterfall Model

- ▶ Assumes requirements are fixed at the start
- ▶ Poor adaptability to changing customer needs
- ▶ Testing and feedback happen late in the process
- ▶ High risk of discovering major flaws late
- ▶ Not well-suited for rapid prototyping or exploratory projects

# Why Agile for Hardware?

- ▶ Traditional hardware development: long, rigid cycles
- ▶ Software has embraced Agile:
  - ▶ Quick iterations
  - ▶ Test-driven development
  - ▶ Continuous integration
- ▶ Hardware complexity is increasing → need for agility!

# Agile Iteration Cycle



# Hardware Design Today

- ▶ VHDL / Verilog = rigid, low-level
- ▶ Hard to reuse and test modularly
- ▶ Long simulation cycles
- ▶ Not optimized for iteration or testing early
- ▶ Often follows a version of the waterfall model

# Software-Inspired Hardware Flow

1. Write modular, parameterized designs
2. Test-first using simulation
3. Version control and continuous integration (CI)
  - ▶ For example, using GitHub Actions
4. Frequent review and refactor
  - ▶ With enough tests, refactoring is safe
5. Code reviews and pull requests

# A Bit More Chisel

- ▶ We had some in the first 30'
- ▶ Here are some more
- ▶ Enough to start generators

# Scala List for Enumeration

```
val empty :: full :: Nil = Enum(2)
```

- ▶ Can be used in wires and registers
- ▶ Symbols for a state machine

# Finite State Machine

```
val empty :: full :: Nil = Enum(2)
val stateReg = RegInit(empty)
val dataReg = RegInit(0.U(size.W))

when(stateReg === empty) {
  when(io.enq.write) {
    stateReg := full
    dataReg := io.enq.din
  }
}.elsewhen(stateReg === full) {
  when(io.deq.read) {
    stateReg := empty
  }
}
```

- A simple buffer for a bubble FIFO

# Parameterization

```
class ParamChannel(n: Int) extends Bundle {  
  val data = Input(UInt(n.W))  
  val ready = Output(Bool())  
  val valid = Input(Bool())  
}
```

```
val ch32 = new ParamChannel(32)
```

- ▶ Bundles and modules can be parametrized
- ▶ Pass a parameter in the constructor

## A Module with a Parameter

```
class ParamAdder(n: Int) extends Module {  
  val io = IO(new Bundle {  
    val a = Input(UInt(n.W))  
    val b = Input(UInt(n.W))  
    val result = Output(UInt(n.W))  
  })  
  
  val addVal = io.a + io.b  
  io.result := addVal  
}  
  
val add8 = Module(new ParamAdder(8))
```

- ▶ Parameter can also be a Chisel type
- ▶ Can also be a generic type:  
▶ `class Mod[T <: Bits](param: T) extends...`

# Factory Methods

- ▶ Simpler component creation and use
- ▶ Usage similar to built in components, such as Mux

```
val myAdder = Adder(x, y)
```

- ▶ A little bit more work on component side
- ▶ Define an apply method on the companion object that returns the component

```
object Adder {  
  def apply(a: UInt, b: UInt) = {  
    val adder = Module(new Adder)  
    adder.io.a := a  
    adder.io.b := b  
    adder.io.result  
  }  
}
```

## More on Testing

- ▶ Using test vectors and `expect()` does not scale
- ▶ Better have co-simulation against a *golden model*
- ▶ Then we can compare
- ▶ Which input values to use?

## A Tiny ALU: IO Connection

```
class Alu extends Module {  
  val io = IO(new Bundle {  
    val fn = Input(UInt(2.W))  
    val a = Input(UInt(4.W))  
    val b = Input(UInt(4.W))  
    val result = Output(UInt(4.W))  
  })
```

```
// Use shorter variable names  
val fn = io.fn  
val a = io.a  
val b = io.b
```

## A Tiny ALU: The Function

```
val result = Wire(UInt(4.W))  
// some default value is needed  
result := 0.U  
  
// The ALU selection  
switch(fn) {  
  is(0.U) { result := a + b }  
  is(1.U) { result := a - b }  
  is(2.U) { result := a | b }  
  is(3.U) { result := a & b }  
}  
  
// Output on the LEDs  
io.result := result  
}
```

# Testing the ALU

- Compute the expected result in Scala

```
// This is exhaustive testing,  
// which usually is impossible  
for (a <- 0 to 15) {  
  for (b <- 0 to 15) {  
    for (op <- 0 to 3) {  
      val result =  
        op match {  
          case 0 => a + b  
          case 1 => a - b  
          case 2 => a | b  
          case 3 => a & b  
        }  
      val resMask = result & 0x0f
```

# Testing the ALU

- ▶ Compare the Scala computed result with the hardware result

```
    poke(dut.io.fn, op)
    poke(dut.io.a, a)
    poke(dut.io.b, b)
    step(1)
    expect(dut.io.result, resMask)
  }
}
```

# Printf Debugging

- ▶ We can *print* in the hardware
- ▶ Printing happens on the rising edge of the clock
- ▶ Good to see many parallel signals and registers
- ▶ `printf` anywhere in the module definition

```
class DeviceUnderTestPrintf extends Module {  
  val io = IO(new Bundle {  
    val a = Input(UInt(2.W))  
    val b = Input(UInt(2.W))  
    val out = Output(UInt(2.W))  
  })  
  
  io.out := io.a & io.b  
  printf("dut: %d %d %d\n", io.a, io.b, io.out)  
}
```

# Test Driven Development (TDD)

- ▶ Software development process
  - ▶ Can we learn from SW development for HW design?
- ▶ Writing the test first, then the implementation
- ▶ Started with extreme programming
  - ▶ Frequent releases
  - ▶ Accept change as part of the development
- ▶ A path to *Agile Hardware Development!*
- ▶ Not used in its pure form
  - ▶ Writing all those tests is simply considered too much work

# Continuous Integration

- ▶ Run your tests on each change
- ▶ Do it also when using source control
- ▶ GitHub Actions
- ▶ I am doing it even for the Chisel book

# Testing versus Debugging

- ▶ Debugging is during code development
- ▶ Waveform and println are easy tools for debugging
- ▶ Debugging does not help for regression tests
- ▶ Write small test cases for regression tests
- ▶ Keeps your code base *intact* when doing changes
- ▶ Better confidence in changes not introducing new bugs

# Chisel Summary

- ▶ Chisel is a small language
- ▶ Embedding it in Scala gives it power
- ▶ We can write circuit generators
- ▶ We can do co-simulation
- ▶ We will continue with Scala

# Is it Time for a Break?

- ▶ Fill up with coffee

# Chisel and Scala

- ▶ Chisel is a library written in Scala
  - ▶ Import the library with `import chisel3._`
- ▶ Chisel code is Scala code
- ▶ When it is run is *generates* hardware
  - ▶ Verilog for synthesis and simulation
- ▶ Chisel is an embedded domain-specific language
- ▶ Two languages in one can be a little bit confusing

# Scala

- ▶ Object-oriented
- ▶ Functional
- ▶ Strongly typed with very good type inference
- ▶ Runs on the Java virtual machine
- ▶ Can call Java libraries
- ▶ Consider it as Java++
  - ▶ Can almost be written like Java
  - ▶ With a more lightweight syntax
  - ▶ Compiled to the JVM
  - ▶ Good Java interoperability
  - ▶ Many libraries available
- ▶ <https://docs.scala-lang.org/tour/tour-of-scala.html>

# Scala Hello World

```
object HelloWorld extends App {  
  println("Hello, World!")  
}
```

- ▶ Compile with `scalac` and run with `scala`
- ▶ Or with `sbt run`
- ▶ You can even use Scala as a scripting language
- ▶ `scala-cli` is a generic Scala runner
- ▶ Show both
- ▶ Use `scala-cli` locally along the examples presented

# Scala Values and Variables

*// A value is a constant*

```
val i = 0
```

*// No new assignment; this will not compile*

```
i = 3
```

*// A variable can change the value*

```
var v = "Hello"
```

```
v = "Hello World"
```

*// Type usually inferred, but can be declared*

```
var s: String = "abc"
```

# Simple Loops

```
// Loops from 0 to 9  
// Automatically creates loop value i  
for (i <- 0 until 10) {  
    println(i)  
}
```

# Conditions

```
for (i <- 0 until 10) {  
  if (i%2 == 0) {  
    println(i + " is even")  
  } else {  
    println(i + " is odd")  
  }  
}
```

## Scala Arrays and Lists

```
// An integer array with 10 elements  
val numbers = new Array[Integer](10)  
for (i <- 0 until numbers.length) {  
    numbers(i) = i*10  
}  
println(numbers(9))
```

```
// List of integers  
val list = List(1, 2, 3)  
println(list)  
// Different form of list construction  
val listenum = 'a' :: 'b' :: 'c' :: Nil  
println(listenum)
```

# Scala Classes

```
// A simple class
class Example {
  // A field, initialized in the constructor
  var n = 0

  // A setter method
  def set(v: Integer) = {
    n = v
  }

  // Another method
  def print() = {
    println(n)
  }
}
```

# Scala (Singleton) Object

```
object Example {}
```

- ▶ For *static* fields and methods
  - ▶ Scala has no static fields or methods like Java
- ▶ Needed for `main`
- ▶ Useful for helper functions

# Tuples

- ▶ Scala has the notion of tuples
- ▶ Can hold a sequence of different types
- ▶ Fields are then accessed with `._n`, starting with 1
- ▶ Easy option to return more than one value from a function

```
val city = (2000, "Frederiksberg")  
val zipCode = city._1  
val name = city._2
```

# Scala Collections

- ▶ Scala has a powerful [collection library](#)
- ▶ Seq is an ordered collection of elements (also called a sequence)
- ▶ The default implementation is immutable
- ▶ We index into a Seq with (), with zero-based indexing.

```
val numbers = Seq(1, 15, -2, 0)
val second = numbers(1)
```

# Functional Programming

- ▶ Functional programming (FP) treats computation as the evaluation of functions
- ▶ Functions are first-class objects
- ▶ Can be a parameter of a function
- ▶ Can be returned from a function
- ▶ Avoid mutable state
- ▶ Recursion is *not* the main point of FP
- ▶ Higher-order functions take functions as arguments

# First-Class Functions

- ▶ In Scala, functions can be assigned to variables, passed as arguments, or returned

```
// A normal function definition  
def double(x: Int): Int = x * 2
```

```
println(double(5))    // prints 10
```

```
// A function that takes another function  
def applyTwice(f: Int => Int, v: Int): Int = {  
    f(f(v))  
}
```

```
println(applyTwice(double, 3)) // prints 12
```

## Function Literals (Anonymous Functions)

- ▶ A function literal is a shorthand for defining a function "inline"
- ▶ by the `=>` symbol
- ▶ Does not require a name (`def`)
- ▶ Often used with higher-order functions

```
// Normal function
```

```
def square(x: Int): Int = x * x
```

```
// Function literal (anonymous function)
```

```
val squareFn = (x: Int) => x * x
```

```
// Using a literal directly
```

```
val nums = List(1, 2, 3, 4)
```

```
val squares = nums.map(x => x * x)
```

```
println(squares) // List(1, 4, 9, 16)
```

# Higher-Order Functions

- ▶ Functions that take other functions as arguments
- ▶ What we just used before
- ▶ Useful for hardware generators (map, filter, etc.)

```
val nums = List(1, 2, 3, 4)
val squares = nums.map(x => x * x)
println(squares) // List(1, 4, 9, 16)
```

# Immutability

- ▶ FP encourages immutable values (using `val`)
- ▶ No side effects: easier to reason about hardware generation

```
val a = 5  
// a = 6    // ERROR: reassignment not allowed
```

```
val b = a + 1    // OK, creates a new value
```

- ▶ In Chisel, `val` describes structure, not time-varying state
- ▶ Registers (`Reg`) are explicit when you need mutable hardware state

# Mapping Over Collections

- ▶ `map` applies a function to each element of a collection
- ▶ Produces a new collection of the same size

```
val nums = List(1, 2, 3, 4)
def square(x: Int): Int = x * x

val squares = nums.map(square)
println(squares) // List(1, 4, 9, 16)
```

# Filtering Collections

- ▶ `filter` keeps only elements that satisfy a condition

```
val nums = List(1, 2, 3, 4, 5, 6)
```

```
def isEven(x: Int): Boolean = (x % 2 == 0)
```

```
val evens = nums.filter(isEven)
```

```
println(evens) // List(2, 4, 6)
```

# Reducing Collections

- ▶ reduce combines all elements using a binary function
- ▶ Useful for sums, products, and combining signals

```
val nums = List(1, 2, 3, 4)
```

```
def add(x: Int, y: Int): Int = x + y
```

```
val sum = nums.reduce(add)
```

```
println(sum) // 10
```

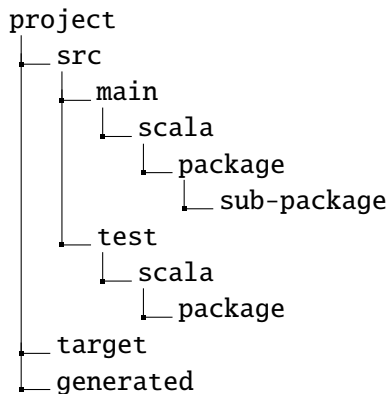
- ▶ Hardware analogy: adding a set of signals
- ▶ Use `reduceTree` for a balanced reduction tree

# Scala Build Tool (sbt)

- ▶ Downloads Scala compiler if needed
- ▶ Downloads dependent libraries (e.g., Chisel)
- ▶ Compiles Scala programs
- ▶ Executes Scala programs
- ▶ Does a lot of magic, maybe too much
- ▶ Compile and run with:

```
sbt run
```

# File Organization in Scala/Chisel



# ScalaTest

- ▶ Testing framework for Scala and Java
- ▶ sbt understands ScalaTest
- ▶ Add library to build.sbt

```
libraryDependencies += "org.scalatest" %%  
    "scalatest" % "3.1.4" % "test"
```

- ▶ Run all tests with:

```
sbt test
```

- ▶ When all (unit) tests are ok, the test passes
- ▶ A little bit funny syntax
- ▶ ChiselTest is based on ScalaTest

# ScalaTest Hello World

```
import org.scalatest._
import org.scalatest.flatspec.AnyFlatSpec
import org.scalatest.matchers.should.Matchers

class ExampleTest extends AnyFlatSpec with
  Matchers {
  "Integers" should "add" in {
    val i = 2
    val j = 3
    i + j should be (5)
  }

  "Integers" should "multiply" in {
    val a = 3
    val b = 4
    a * b should be (12)
  }
}
```

# Further Reading and Web Resources

- ▶ [Scala defined hardware generators for Chisel](#)
  - ▶ Journal article with generator examples
- ▶ [Chisel book website](#)
  - ▶ Information on Chisel, a bit of Scala
  - ▶ Download the free PDF
- ▶ [Digital design course at DTU](#)
  - ▶ Slides on digital design with Chisel
- ▶ [Digital design lab at DTU](#)
  - ▶ Lab material for the digital design course
  - ▶ Option to train a bit on Chisel

# Lab 1

- ▶ I assume that you installed all tools during lab0
- ▶ Functional programming in Scala
- ▶ The lab contains tests; run with `sbt test`
- ▶ Code is in [lab1](#)

# Session 3

- ▶ Introduction to Chisel
- ▶ Functional programming for generators

# Chisel in Scala

- ▶ Chisel components are Scala classes
- ▶ Chisel code is in the constructor
- ▶ Executed at object creation time
- ▶ Builds the network of hardware objects
- ▶ Testers are written in Scala to drive the tests

# Chisel Data Types

- ▶ Data types for values on wires or state elements
- ▶ Raw collection of bits is type `Bits`
- ▶ Simple types to represent integer numbers
  - ▶ Unsigned: `UInt`
  - ▶ Signed: `SInt`
  - ▶ Subtype of `Bits`
- ▶ Automatic bit width inference
- ▶ Boolean values are of type `Bool`, a single bit value
- ▶ *Interesting* way to specify constants

```
1.U  
"habcd".U  
"b0101".U  
-5.S  
true.B
```

# Chisel Data Types

- ▶ Bit width can be explicitly specified with a width type
  - ▶ SInt will be sign extended
  - ▶ UInt will be zero extended

UInt(8.W)

SInt(16.W)

0.U(32.W)

"abcd".U(24.W)

-5.S(16.W)

- ▶ Chisel data types are different from Scala builtin types (e.g., Scala's Int)

# Bitwise Logical Operations

- ▶ Bitwise NOT, AND, OR, and XOR
- ▶ Automatic size extension to larger operand

```
val notVal = ~x
val maskOut = x & "b00001111".U
val orVal = x | y
val xorVal = x ^ y
```

- ▶ Bit reduction
- ▶ Results in a single bit

```
x.andR
x.orR
x.xorR
```

# Arithmetic Operations

- ▶ Addition, subtraction, multiplication, division, modulus
- ▶ Automatic size extension to larger operand

$+$ ,  $-$ ,  $*$ ,  $/$ ,  $\%$

- ▶ Left and right shifts
- ▶ Left shift extends bit width
- ▶ Right shift reduces bit width

$\ll$ ,  $\gg$

# Comparison

- ▶ The usual operations
  - ▶ Unusual equal and unequal operator symbols
  - ▶ To keep the original Sala operators usable for references
- ▶ Operands are UInt and SInt
- ▶ Operands can be Bool for equal and unequal
- ▶ Result is Bool

===, /=

>, >=, <, <=

# Boolean Logical Operations

- ▶ Operands and result are Bool
- ▶ Logical NOT, AND, and OR

```
val notX = !x  
val bothTrue = a && b  
val orVal = x || y
```

# Combinational Circuits

- ▶ Circuit is a graph of nodes
- ▶ A node is a hardware operator with zero or more inputs
- ▶ Textual expression to wire up nodes
- ▶ Named wires with some (unspecified) width

`(a | b) & ~c`

# Combinational Circuits

- ▶ Simple expressions represent a circuit tree
- ▶ Arbitrary directed acyclic graphs need named subexpressions
- ▶ Using Scala's `val` keyword for variables that don't change
- ▶ Referenced multiple times

```
val cond = a & b  
val result = (cond & selA) | (!cond & selB)
```

# Connections

- ▶ Connections with the `:=` assignment
- ▶ When *reassigning* a value to a wire, port, or register

```
adder.io.a := ina  
adder.io.b := inb
```

# Register

- ▶ State elements
- ▶ Has it's own Chisel type `Reg`
- ▶ Positive edge triggered D flip-flop
- ▶ Synchronous reset
- ▶ Clock and reset are *hidden wires*

```
val q = RegNext(d)
```

- ▶ `d` is the input, `q` the output
- ▶ Register type is inferred by the input (`d`) type

# Register

- ▶ Reset value as parameter on a `RegInit` constructor

```
val initReg = RegInit(0.U(8.W))
```

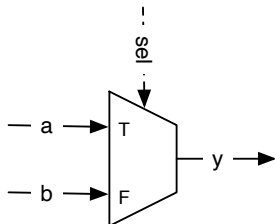
- ▶ With this forward declaration we later assign the input

```
initReg := initReg + 1.U
```

- ▶ A register can also be defined within an expression

```
val risingEdge = din & !RegNext(din)
```

# A Multiplexer



- ▶ A Multiplexer selects between alternatives
- ▶ So common that Chisel provides a construct for it
- ▶ Selects `a` when `sel` is `true`. `b` otherwise
- ▶ Could be implemented with conditional updates
- ▶ Automagical type selection on input types

```
val result = Mux(sel, a, b)
```

# Conditional Updates for Combinational Circuits

```
val w = Wire(UInt())

when (cond) {
  w := 1.U
} .elsewhen (cond2) {
  w := 2.U
} .otherwise {
  w := 3.U
}
```

- ▶ Similar to VHDL process or SystemVerilog `always_comb`
- ▶ Chisel checks for complete assignments in all branches
- ▶ Latches give compile error

# Chained Conditionals

- ▶ Chain of conditionals with `.elsewhen`
- ▶ With an optional *else* path with `.otherwise`
- ▶ Note that Scala has `if/else`
  - ▶ Does NOT result in hardware
  - ▶ Are used to conditionally *generate* hardware
  - ▶ We will look at this later
- ▶ Note the “.” at the operators

```
when (c1) { v := 1.U }  
.elsewhen (c2) { v := 2.U }  
.otherwise { v := 3.U }
```

# Switch Statement

- ▶ Series of comparisons
- ▶ Chisel allows combinational logic be updated conditionally
- ▶ Chisel disallows incomplete specified logic (= latches)
- ▶ Chisel will report a runtime error

```
switch(fn) {  
  is(0.U) { result := a + b }  
  is(1.U) { result := a - b }  
  is(2.U) { result := a | b }  
  is(3.U) { result := a & b }  
}
```

# Data Aggregation with Bundles

- ▶ A Bundle groups several named fields
- ▶ Like a C struct or VHDL record
- ▶ Vec is a vector of elements with the same type
- ▶ Can be arbitrary mixed

```
class AluFields extends Bundle {  
  val function = UInt(2.W)  
  val inputA = UInt(8.W)  
  val inputB = UInt(8.W)  
  val result = UInt(8.W)  
}
```

# Vectors

- ▶ Indexable vector of elements
- ▶ Elements can be Chisel basic elements, or bundles
- ▶ Type is specified as second parameter

```
val myVec = Vec(3, SInt(10.W))  
val y = myVec(2)  
myVec(0) := -3.S
```

- ▶ A register file as a register of a vector

```
val vecReg = Reg(Vec(32, SInt(32.W)))
```

# IO Ports

```
class Channel extends Bundle {  
  val data = Input(UInt(8.W))  
  val ready = Output(Bool())  
  val valid = Input(Bool())  
}
```

- ▶ Ports are Bundles with directions
- ▶ Ports are used to connect modules
- ▶ Can be reversed with the Flipped
- ▶ Convenient to have one bundle definition working as a source and the destination used between two modules

```
class ChannelUsage extends Bundle {  
  val input = new Channel()  
  val output = Flipped(new Channel())  
}
```

# Modules

- ▶ Modules are used to organize the circuit
- ▶ Similar to VHDL components (entity/architecture)
- ▶ A class that inherits from `Module`
- ▶ Circuit description in the constructor
- ▶ Interface (port) is a `Bundle`, wrapped into an `IO()`, and stored in the field `io`

```
class Adder extends Module {  
  val io = IO(new Bundle {  
    val a = Input(UInt(4.W))  
    val b = Input(UInt(4.W))  
    val result = Output(UInt(4.W))  
  })  
  
  val addVal = io.a + io.b  
  io.result := addVal  
}
```

# Module Usage

- ▶ Create with `new` and wrap into a `Module()`
- ▶ Interface port via the `io` field
- ▶ Note the assignment operator `:=` on `io` fields

```
val adder = Module(new Adder())  
adder.io.a := ina  
adder.io.b := inb  
val result = adder.io.result
```

# Hello World in Chisel

```
class Hello extends Module {  
  val io = IO(new Bundle {  
    val led = Output(UInt(1.W))  
  })  
  val CNT_MAX = (500000000 / 2 - 1).U  
  
  val cntReg = RegInit(0.U(32.W))  
  val blkReg = RegInit(0.U(1.W))  
  
  cntReg := cntReg + 1.U  
  when(cntReg === CNT_MAX) {  
    cntReg := 0.U  
    blkReg := ~blkReg  
  }  
  io.led := blkReg  
}
```

# Chisel Main

- ▶ Create one top-level Module
- ▶ Invoke the Verilog emitter from the Scala `main` (or `App`)
- ▶ The following code generates Verilog code

```
object Hello extends App {  
    emitVerilog(new Hello())  
}
```

# Time for a Break?

- ▶ We need some coffee and sweet stuff

# Functions

- ▶ Circuits can be encapsulated in functions
- ▶ Each *function call* generates hardware
- ▶ Simple functions can be a single line

```
def adder(v1: UInt, v2: UInt) = v1 + v2
```

```
val add1 = adder(a, b)
```

```
val add2 = adder(c, d)
```

# Functional Abstraction

```
def addSub(add: Bool, a: UInt, b: UInt) =  
  Mux(add, a+b, a-b)
```

```
val res = addSub(cond, a, b)
```

```
def rising(d: Bool) = d && !RegNext(d)
```

- ▶ Functions for repeated pieces of logic
- ▶ May contain state
- ▶ Functions may return *hardware*

# The Counter as a Function

- ▶ Longer functions in curly brackets
- ▶ Last value is the return value

```
def counter(n: UInt) = {  
  
    val cntReg = RegInit(0.U(8.W))  
  
    cntReg := cntReg + 1.U  
    when(cntReg == n) {  
        cntReg := 0.U  
    }  
    cntReg  
}  
  
val counter100 = counter(100.U)
```

# Use Functional Programming for Generators

```
def add(a: UInt, b: UInt) = a + b
```

```
val sum = vec.reduce(add)
```

```
val sum = vec.reduce(_ + _)
```

```
val sum = vec.reduceTree(_ + _)
```

- ▶ This is a simple example
- ▶ What about an arbitration tree with fair arbitration?

# Functional Generation

- ▶ Anonymous functions, called *function literal*

```
(param) => function body
```

- ▶ A function for a minimum search

```
val min = vec.reduceTree((x, y) => Mux(x <  
    y, x, y))
```

# Testing with Chisel

- ▶ A test contains:
  - ▶ a device under test (DUT) and
  - ▶ the testing logic
- ▶ Set input values with `poke`
- ▶ Advance the simulation with `step`
- ▶ Read the output values with `peek`
- ▶ Compare the values with `expect`
- ▶ Import following packages:

```
import chisel3._  
import chiseltest._  
import org.scalatest.flatspec.AnyFlatSpec
```

# An Example DUT

- ▶ A device-under test (DUT)
- ▶ Just 2-bit AND logic

```
class DeviceUnderTest extends Module {  
  val io = IO(new Bundle {  
    val a = Input(UInt(2.W))  
    val b = Input(UInt(2.W))  
    val out = Output(UInt(2.W))  
    val equ = Output(Bool())  
  })  
  
  io.out := io.a & io.b  
  io.equ := io.a === io.b  
}
```

# A ChiselTest

- ▶ Extends class `AnyFlatSpec` with `ChiselScalatestTester`
- ▶ Has the device-under test (DUT) as parameter of the test function
- ▶ Test function contains the test code
- ▶ Testing code can use all features of Scala

# A Test

- ▶ Poke values and expect some output

```
class SimpleTestExpect extends AnyFlatSpec
  with ChiselScalatestTester {
    "DUT" should "pass" in {
      test(new DeviceUnderTest) { dut =>
        dut.io.a.poke(0.U)
        dut.io.b.poke(1.U)
        dut.clock.step()
        dut.io.out.expect(0.U)
        dut.io.a.poke(3.U)
        dut.io.b.poke(2.U)
        dut.clock.step()
        dut.io.out.expect(2.U)
      }
    }
  }
}
```

## Lab 2

- ▶ Write a search for the maximum circuit (with `treeReduce()`)
- ▶ Optional: add the generation of the index of the maximum value
- ▶ Emit Verilog with `emitVerilog(new Comp())`
- ▶ Code is in [lab2](#)

# Session 4

- ▶ Hardware generators
- ▶ Object-oriented hardware design

# Scala List for Enumeration

```
val empty :: full :: Nil = Enum(2)
```

- ▶ Can be used in wires and registers
- ▶ Symbols for a state machine

# Finite State Machine

```
val empty :: full :: Nil = Enum(2)
val stateReg = RegInit(empty)
val dataReg = RegInit(0.U(size.W))

when(stateReg === empty) {
  when(io.enq.write) {
    stateReg := full
    dataReg := io.enq.din
  }
}.elsewhen(stateReg === full) {
  when(io.deq.read) {
    stateReg := empty
  }
}
```

- A simple buffer for a bubble FIFO

# Component Generation

```
val cores = new Array[Module](32)
```

```
for (j <- 0 until 32)  
  cores(j) = Module(new CPU())
```

- ▶ Use Scala array to collect components
- ▶ Generation with a Scala loop

# Logic Generation

- ▶ Read a file into a table
  - ▶ E.g., to read in ROM content for a processor
- ▶ Generate a truth table algorithmically
  - ▶ E.g., generate binary to BCD translation
- ▶ Use the full power of Scala

```
val byteArray =  
    Files.readAllBytes(Paths.get(fileName))  
val arr = new Array[Bits](byteArray.length)  
for (i <- 0 until byteArray.length) {  
    arr(i) = Bits(byteArray(i), 8)  
}  
val rom = Vec[Bits](arr)
```

# Parameterization

```
class ParamChannel(n: Int) extends Bundle {  
  val data = Input(UInt(n.W))  
  val ready = Output(Bool())  
  val valid = Input(Bool())  
}
```

```
val ch32 = new ParamChannel(32)
```

- ▶ Bundles and modules can be parametrized
- ▶ Pass a parameter in the constructor

## A Module with a Parameter

```
class ParamAdder(n: Int) extends Module {  
  val io = IO(new Bundle {  
    val a = Input(UInt(n.W))  
    val b = Input(UInt(n.W))  
    val result = Output(UInt(n.W))  
  })  
  
  val addVal = io.a + io.b  
  io.result := addVal  
}  
  
val add8 = Module(new ParamAdder(8))
```

- ▶ Parameter can also be a Chisel type
- ▶ Can also be a generic type:
- ▶ `class Mod[T <: Bits](param: T) extends...`

# Scala for Loop for Circuit Generation

```
val shiftReg = RegInit(0.U(8.W))

shiftReg(0) := inVal

for (i <- 1 until 8) {
  shiftReg(i) := shiftReg(i-1)
}
```

- ▶ for is Scala
- ▶ This loop generates several connections
- ▶ The connections are parallel hardware

# Conditional Circuit Generation

```
class Base extends Module { val io = new Bundle() }  
class VariantA extends Base { }  
class VariantB extends Base { }
```

```
val m = if (useA) Module(new VariantA())  
        else Module(new VariantB())
```

- ▶ if and else is Scala
- ▶ if is an expression that returns a value
  - ▶ Like “cond ? a : b;” in C and Java
- ▶ This is not a hardware multiplexer
- ▶ Decides which module to generate
- ▶ Could even read an XML file for the configuration

# Generics in Hardware Construction

- ▶ Chisel supports generic classes with type parameters
- ▶ Write hardware generators independent of concrete type
- ▶ We have seen this in action with the Chisel Mux
- ▶ This is a multiplexer *generator*

```
def myMux[T <: Data](sel: Bool, tPath: T, fPath:
  T): T = {

  val ret = WireDefault(fPath)
  when (sel) {
    ret := tPath
  }
  ret
}
```

# Put Generics Into Use

- ▶ Let us implement a generic FIFO
- ▶ Use the generic ready/valid interface from Chisel

```
class DecoupledIO[T <: Data](gen: T) extends
  Bundle {
    val ready = Input(Bool())
    val valid = Output(Bool())
    val bits  = Output(gen)
  }
```

# Define the FIFO Interface

```
class FifoIO[T <: Data](private val gen: T)
  extends Bundle {
    val enq = Flipped(new DecoupledIO(gen))
    val deq = new DecoupledIO(gen)
  }
```

- ▶ We need enqueueing and dequeueing ports
- ▶ Note the Flipped
  - ▶ It switches the direction of ports
  - ▶ No more double definitions of an interface

## But What FIFO Implementation?

- ▶ Bubble FIFO (good for low data rate)
- ▶ Double buffer FIFO (fast restart)
- ▶ FIFO with memory and pointers (for larger buffers)
  - ▶ Using flip-flops
  - ▶ Using on-chip memory
- ▶ And some more...
- ▶ This calls for object-oriented programming *hardware construction*

# Abstract Base Class and Concrete Extension

```
abstract class Fifo[T <: Data](gen: T, val depth:
  Int) extends Module {
  val io = IO(new FifoIO(gen))

  require(depth > 0, "Number of buffer elements
    needs to be larger than 0")
}
```

- ▶ May contain common code
- ▶ Extend by concrete classes

```
class BubbleFifo[T <: Data](gen: T, depth: Int)
  extends Fifo(gen: T, depth: Int) {
```

# Select a Concrete FIFO Implementation

- ▶ Decide at hardware generation
- ▶ Can use all Scala/Java power for the decision
  - ▶ Connect to a web service, get Google Alphabet stock price, and decide on which to use ;-)
  - ▶ For sure a silly idea, but you see what is possible...
  - ▶ Developers may find clever use of the Scala/Java power
  - ▶ We could present a GUI to the user to select from
- ▶ We use XML files parsed at hardware generation time
- ▶ End of TCL, Python,... generated hardware

# Combinational (Truth) Table Generation

```
val arr = new Array[Bits](length)
for (i <- 0 until length) {
  arr(i) = ...
}
val rom = Vec[Bits](arr)
```

- ▶ Generate a table in a Scala array
- ▶ Use that array as input for a Chisel Vec
- ▶ Generates a logic table at hardware construction time

# Ideas for Runtime Table Generation

- ▶ Assembler in Scala/Java generates the boot ROM
- ▶ Table with a `sin` function
- ▶ Binary to BCD conversion
- ▶ Schedule table for a TDM-based network-on-chip
- ▶
- ▶ More ideas?

# Test Driven Development (TDD)

- ▶ Software development process
  - ▶ Can we learn from SW development for HW design?
- ▶ Writing the test first, then the implementation
- ▶ Started with extreme programming
  - ▶ Frequent releases
  - ▶ Accept change as part of the development
- ▶ A path to *Agile Hardware Development!*
- ▶ Not used in its pure form
  - ▶ Writing all those tests is simply considered too much work

# Continuous Integration

- ▶ Run your tests on each change
- ▶ Do it also when using source control
- ▶ GitHub Actions
- ▶ I am doing it even for the Chisel book

# Testing versus Debugging

- ▶ Debugging is during code development
- ▶ Waveform and println are easy tools for debugging
- ▶ Debugging does not help with regression tests
- ▶ Write small test cases for regression tests
- ▶ Keeps your code base *intact* when doing changes
- ▶ Better confidence in changes not introducing new bugs

## Lab 3

- ▶ Design a component with a generic parameter
- ▶ Emit Verilog with `emitVerilog(new Comp())`, you can synthesize it then on Thursday
- ▶ Code is in [lab3](#)

# Summary

- ▶ Processors do not get much faster – we need to design custom hardware
- ▶ A great time for hardware developers!
- ▶ Write hardware generators and not just RTL
- ▶ Chisel builds on the power of object-oriented and functional Scala
- ▶ Hope you try to get started with Scala/Chisel generators
- ▶ Thank you for this afternoon!